Tổng hợp kiến thức

Web Design and Programming — Week 01 → Week 07

Tổng hợp từ slide bài giảng của M.Sc. Le Duy Khanh (week01.pdf … week07.pdf)

Tuần	Chủ đề
1	Kiến trúc Web & quản lý phiên bản (Git/GitHub)
2	HTML: Semantic, Tables & Forms
3	CSS3: Box Model, Flexbox/Grid & Responsive
4	JavaScript cơ bản & DOM
5	JavaScript bất đồng bộ & Fetch/AJAX
6	Python Web & bắt đầu với FastAPI
7	FastAPI: Routing & Request/Response

Week 01 — Kiến trúc Web & Version Control

Mục tiêu: hiểu mô hình client–server, vòng đời HTTP, cấu trúc URL và quy trình Git cơ bản.

1. Web hoạt động thế nào

- **Internet**: hạ tầng mạng toàn cầu nối các máy tính. **Web (WWW)**: một dịch vụ chạy trên Internet dùng HTTP/HTTPS (bên cạnh email, SSH, FTP…).
- **Trình duyệt** là client: gửi request và hiển thị (render) tài nguyên web.
- **Client-side (frontend)**: chạy trong trình duyệt — HTML (cấu trúc), CSS (trình bày), JavaScript (hành vi).
- **Server-side (backend)**: nhận request, chạy logic, làm việc với database, gửi response.
- **Vòng đời HTTP**: client gửi **method + URL + headers (+ body)** → server xử lý → trả **status code + headers + nội dung**.
- Trình duyệt tải thêm tài nguyên liên kết (CSS, JS, ảnh, font). Hiệu năng phụ thuộc số request, dung lượng, cache.

2. HTTP methods & status codes

Method	Ý nghĩa	Status code	Ý nghĩa
GET	Lấy tài nguyên	200 OK	Thành công
POST	Tạo mới / gửi dữ liệu	301 / 302	Chuyển hướng (redirect)
PUT / PATCH	Cập nhật	404 Not Found	Không tìm thấy tài nguyên
DELETE	Xoá	500 Internal Server Error	Lỗi phía server

3. Cấu trúc URL

```
https://example.com/courses/week01/index.html
  │          │               └─ path: /courses/week01/index.html
  │          └─ domain name: example.com
  └─ protocol: https
```

- **Absolute path**: bắt đầu từ gốc `/`. **Relative path**: tính từ vị trí trang hiện tại.

4. Git — khái niệm cốt lõi

- **Repository**: cơ sở dữ liệu lịch sử của dự án.
- **Working directory**: các file đang sửa. **Staging area (index)**: tập thay đổi được chọn cho commit tiếp theo.
- **Commit**: ảnh chụp (snapshot) kèm tác giả, thời gian, message. **Branch**: nhánh phát triển độc lập.
- Lợi ích: lưu lịch sử, làm việc nhóm song song, rollback khi có bug, ghi lại quyết định kỹ thuật.

```
git init
git status
git add index.html
git commit -m "Create initial webpage"
git branch feature/navbar
git switch feature/navbar

git remote add origin <repository-url>
git push -u origin main      # đẩy commit lên remote
git pull origin main         # fetch + merge từ remote
```

- Commit sớm, commit thường xuyên, message có ý nghĩa. **Pull trước khi push** để giảm conflict.
- `.gitignore` để không commit thư mục nặng (dataset, `node_modules`, `.venv`).

5. Branch vs Fork vs Pull Request

Khái niệm	Ý nghĩa
Branch	Làm việc song song **trong cùng một repository**
Fork	Bản sao repository của người khác về tài khoản của mình
Pull Request	Yêu cầu merge thay đổi, để review trước khi gộp

Quy trình open-source: Fork → tạo feature branch → commit & push → mở PR để review và merge.

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
Status code nghĩa là thành công?	200
Lệnh đưa thay đổi vào staging?	git add
Lệnh lấy cập nhật từ remote?	git pull
Branch và Fork khác nhau chủ yếu ở?	Phạm vi sở hữu repository
`https` trong URL là thành phần gì?	Protocol

Week 02 — HTML: Semantic, Tables & Forms

HTML = cấu trúc (bộ xương). CSS tạo kiểu (Week 3), JavaScript tạo hành vi (Week 4).

1. Khung tài liệu HTML

```
<!doctype html>
<html lang="vi">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Page Title</title>
</head>
<body>
  <!-- Nội dung hiển thị -->
</body>
</html>
```

- HTML là các **element lồng nhau** tạo thành cây. Element = thẻ mở + nội dung + thẻ đóng; có thể mang **attribute**.
- Thẻ tự đóng (không có nội dung): `<img>`, `<br>`, `<hr>`.
- `lang="vi"` giúp screen reader; `charset="utf-8"` hiển thị đúng tiếng Việt; `viewport` để hiển thị đúng trên mobile.

2. Thẻ nội dung cơ bản

- Heading `<h1>`...`<h6>`: theo thứ bậc ý nghĩa, **không nhảy cấp**.
- `<p>`, `<br>`, `<hr>`; `<strong>` (quan trọng), `<em>` (nhấn mạnh).
- Link `<a href="...">` (ngoài trang hoặc `#id` trong trang); ảnh `<img src alt>` — `alt` **bắt buộc** với ảnh có ý nghĩa.
- Danh sách `<ul>` (không thứ tự), `<ol>` (có thứ tự), `<li>`.
- **Block** (chiếm cả dòng: `div`, `p`, `h1`) vs **inline** (nằm trong dòng: `span`, `a`, `strong`).
- Chỉ dùng `div`/`span` khi không có thẻ ngữ nghĩa phù hợp.

3. Semantic HTML

Chọn thẻ theo **ý nghĩa**, không theo giao diện → tốt cho accessibility, SEO và làm việc nhóm.

```
<body>
  <header>...</header>   <!-- đầu trang: logo, tiêu đề -->
  <nav>...</nav>         <!-- thanh điều hướng -->
  <main>                 <!-- DUY NHẤT một vùng nội dung chính -->
    <section>...</section>
    <article>...</article>
  </main>
  <aside>...</aside>     <!-- nội dung phụ, sidebar -->
  <footer>...</footer>   <!-- cuối trang -->
</body>
```

4. Tables

- Chỉ dùng table cho **dữ liệu dạng bảng** (hàng × cột), **không** dùng để dàn layout.
- Thẻ lõi: `table`, `tr` (hàng), `th` (ô tiêu đề), `td` (ô dữ liệu). Cấu trúc: `caption`, `thead`, `tbody`, `tfoot`.
- Gộp ô: `colspan="n"` (gộp n cột), `rowspan="n"` (gộp n hàng).

```
<table>
  <caption>Model performance comparison</caption>
  <thead><tr><th>Model</th><th>Accuracy</th><th>F1</th></tr></thead>
  <tbody>
    <tr><td>Logistic Regression</td><td>0.91</td><td>0.88</td></tr>
    <tr><td>Random Forest</td><td>0.94</td><td>0.91</td></tr>
  </tbody>
</table>
```

5. Forms & gửi dữ liệu

- `<form>` có 2 attribute quan trọng: `action` (gửi tới endpoint nào) và `method` (GET/POST).
- Control: `input` (type: text, number, email, password, radio, checkbox, date, file), `select/option`, `textarea`, `button type="submit"`.
- `<label for="id">` gắn với `input id` → click label focus input, screen reader đọc được.
- `name` **là attribute quan trọng nhất**: là key trong cặp key–value gửi lên server. **Không có** `name` **= dữ liệu không được gửi.**

```
<form action="/predict" method="post">
  <label for="address">Address</label>
  <input id="address" name="address" type="text" required>
  <label for="bedrooms">Bedrooms</label>
  <select id="bedrooms" name="bedrooms">
    <option value="1">1</option><option value="2">2</option>
  </select>
  <button type="submit">Predict</button>
</form>
```

	GET	POST
Dữ liệu nằm ở	URL (query string)	Request body
Phù hợp cho	Tìm kiếm / lọc	Dữ liệu nhạy cảm / lớn
Bookmark, chia sẻ được	Có	Không
Ví dụ	Trang kết quả tìm kiếm	Đăng nhập / tạo mới / upload

- Validation phía client: `required`, `type="email"`, `min`/`max`, `pattern` — tiện cho người dùng nhưng **không đáng tin**; server phải luôn validate lại.

6. Accessibility — 5 thói quen

- `lang` trên thẻ `<html>`.
- `alt` cho mọi ảnh có ý nghĩa.
- `label` cho mọi input (for/id).
- Heading đúng thứ bậc, không nhảy cấp.
- Độ tương phản chữ/nền đủ lớn.

Week 03 — CSS3: Layout, Box Model & Responsive Design

CSS là hệ thống luật: selector, specificity, cascade. Box Model + Flexbox chiếm ~80% nhu cầu thực tế.

1. CSS cơ bản & selectors

- Rule = **selector** + khối khai báo `property: value;`. Có thể viết inline, trong `<style>`, hoặc file `.css` ngoài (**nên dùng**): `<link rel="stylesheet" href="style.css">`.
- Element selector `p`, class `.card` (nhiều phần tử), ID `#header` (một phần tử duy nhất).
- Kết hợp: descendant `.dashboard .card`, pseudo-class `.card:hover`, `:focus`, nhiều đích `h1, h2, h3`.

2. Cascade, Specificity, Inheritance

- **Cascade**: khi luật xung đột, trình duyệt chọn theo độ ưu tiên, sau đó theo thứ tự trong mã nguồn.
- **Specificity**: inline style > `#id` > `.class` > element. Bằng nhau → luật **viết sau** thắng.
- **Inheritance**: một số thuộc tính (`font-family`, `color`) tự truyền từ cha xuống con.

```
h1      { color: #333; }     /* specificity 0,0,1 */
.title  { color: #0055aa; }  /* specificity 0,1,0 */
#header { color: #cc3300; }  /* specificity 1,0,0  -> THẮNG */
```

3. Box Model (chủ đề thi thường gặp)

Mỗi phần tử là một hộp gồm 4 lớp từ trong ra ngoài: **content → padding → border → margin**.

```
content-box (mặc định):
  Total width = content + 2×padding + 2×border + 2×margin
  VD: width 200, padding 10, border 2, margin 5
      = 200 + 20 + 4 + 10 = 234px

/* Nên áp dụng cho mọi project */
*, *::before, *::after { box-sizing: border-box; }
/* border-box: padding + border nằm TRONG width đã khai báo */
```

	Padding	Margin
Vị trí	Bên trong border	Bên ngoài border
Nền	Mang màu nền của phần tử	Luôn trong suốt
Dùng để	Khoảng cách bên trong	Khoảng cách/tách giữa các phần tử

4. Flexbox & Grid

- **Flexbox**: bố cục **1 chiều** — hàng card, navbar, dòng form. **Grid**: bố cục **2 chiều** — vùng trang, gallery, admin panel.
- Flex container: `display: flex`. Trục chính (mặc định ngang) → `justify-content`; trục phụ (dọc) → `align-items`; `flex-wrap: wrap`; `gap`.
- `justify-content`: flex-start, center, flex-end, space-between, space-around. `align-items`: stretch, center, flex-start, flex-end.
- Flex item: `flex: 1 1 220px` = flex-grow 1, flex-shrink 1, flex-basis 220px.
- Grid: `grid-template-columns: 1fr 1fr 1fr` (3 cột bằng nhau; `1fr` = 1 phần không gian còn trống).

```
.dashboard { display: flex; flex-wrap: wrap; gap: 12px; }
.card      { flex: 1 1 220px; padding: 14px;
             border: 1px solid #d4d4d4; border-radius: 10px; }
.layout    { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 16px; }
```

5. Responsive Web Design

- Một HTML + một CSS tự thích nghi với mọi kích thước màn hình; dùng **đơn vị tương đối** + **media query**, tránh width px cố định cho container ngoài.
- **Viewport meta tag bắt buộc**, nếu thiếu, trình duyệt mobile giả làm desktop và thu nhỏ trang.
- Đơn vị: `rem` (theo font gốc), `%` / `vw` (theo container/viewport). Ảnh: `img { max-width: 100%; height: auto; }`.

```
@media screen and (max-width: 768px) {   /* tablet trở xuống */
  .dashboard { flex-direction: column; }
}
@media screen and (max-width: 480px) {   /* điện thoại */
  .card { padding: 10px; }
}
```

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
Thuộc tính tạo khoảng cách bên trong border?	padding
width 200, padding 10, border 2, margin 5 → tổng?	234px
`.title` hay `#header` thắng?	#header
Media query cho màn hình ≥ 800px?	@media (min-width: 800px)
Thuộc tính phân bố theo trục chính của Flexbox?	justify-content
Vì sao dùng border-box?	Padding + border tính trong width

Week 04 — JavaScript cơ bản & DOM

Tư duy: Python → JavaScript, cùng logic, khác dấu câu. DOM là một cây giống JSON/dict lồng nhau.

1. Nhúng JavaScript

- Embedded `<script>...</script>` hoặc external `<script src="app.js"></script>` (**nên dùng**, tái sử dụng được).
- Đặt script **ngay trước** `</body>` hoặc dùng `defer`. Nếu script chạy trước khi DOM được dựng thì `getElementById()` không tìm thấy phần tử.

```
<script src="app.js" defer></script>
```

2. Python → JavaScript

Python	JavaScript	Ghi chú
x = 5	let x = 5;	Gán lại được
—	const x = 5;	Không gán lại được
—	var x = 5;	Kiểu cũ, tránh dùng
None	undefined / null	undefined = khai báo chưa gán
int / float	Number	
str / bool	String / Boolean	
list / dict	Array / Object	Chính là cấu trúc của JSON
def f(x):	function f(x) { }	
lambda x: x*2	(x) => x * 2	Arrow function

```
"10" == 10    // true  (so sánh sau khi ép kiểu)
"10" === 10   // false (giá trị VÀ kiểu phải khớp)
// Quy tắc: luôn dùng === và !==

if (bedrooms > 3) { console.log("Large"); } else { console.log("Standard"); }
for (let i = 0; i < 5; i++) { console.log(i); }

let scores = [88, 92, 79];                       // Array  ≈ list
let sample = { id: 1, name: "House A" };         // Object ≈ dict
scores[0]; sample.name;
```

3. DOM — Document Object Model

- Trình duyệt biến HTML thành **cây node**; mỗi thẻ, attribute, text đều là node mà JS đọc/sửa được.
- Chọn phần tử: `getElementById()`, `getElementsByClassName()`, `querySelector()` (phần tử khớp đầu tiên), `querySelectorAll()` (tất cả). Nên dùng **querySelector** vì cú pháp giống CSS selector.
- Đọc/sửa: `innerHTML`, `textContent`, `getAttribute()`, `setAttribute()`, `style.color`. Thuộc tính CSS có gạch nối → camelCase: `background-color` → `style.backgroundColor`.
- Tạo/xoá: `createElement()`, `appendChild()`, `removeChild()`, giúp vẽ nội dung mới mà không cần tải lại trang.

4. Events & validate form phía client

```
let form = document.querySelector("#house-form");
form.addEventListener("submit", function (event) {
  event.preventDefault();                 // chặn reload / gửi mặc định
  let bedrooms = document.querySelector("#bedrooms").value;
  if (bedrooms === "" || Number(bedrooms) <= 0) {
    let error = document.createElement("p");
    error.textContent = "Bedrooms must be a positive number.";
    form.appendChild(error);
  } else {
    console.log("Form is valid - ready to send.");
  }
});
```

- `addEventListener()` là cách hiện đại: tách hành vi khỏi HTML, gắn được nhiều listener (khác `onclick="..."`).
- `event.target` là phần tử gây ra sự kiện. Event hay dùng: click, submit, change, keyup, focus, blur.
- Validate phía client để báo lỗi ngay, không cần gọi server (nhưng server vẫn phải validate lại).

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
`"10" == 10` và `"10" === 10`?	true, false
let khác const?	const không gán lại được
Method DOM dùng CSS selector?	querySelector / querySelectorAll
Cách gắn nhiều handler cho 1 phần tử?	addEventListener()
Chặn submit/reload mặc định của form?	event.preventDefault()
Giá trị biến khai báo nhưng chưa gán?	undefined

Week 05 — JavaScript bất đồng bộ & Fetch/AJAX

JSON là cầu nối giữa frontend và mọi backend / mô hình AI.

1. Vì sao cần bất đồng bộ

- JS chạy **đơn luồng**: một main thread vừa chạy JS vừa render UI. Tác vụ chậm (gọi mạng) chạy đồng bộ sẽ **đóng băng** trang.
- Giải pháp: giao tác vụ chậm cho trình duyệt chạy nền, JS tiếp tục chạy, xử lý kết quả khi xong, do **Event Loop** điều phối.
- So sánh: `requests.get()` của Python thường là blocking (không có UI để đóng băng); `fetch()` trong trình duyệt thì non-blocking.

2. Promise

Trạng thái	Ý nghĩa
pending	Đang chờ thao tác hoàn thành
fulfilled	Thành công, có kết quả
rejected	Thất bại, có lỗi

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())   // chạy khi fulfilled
  .then(data => console.log(data))
  .catch(error => console.error(error)); // chạy khi rejected
```

3. async / await + try/catch

- `async function` **luôn trả về Promise**. `await` chỉ dùng được **bên trong async function**, tạm dừng đến khi Promise xong, nên code đọc từ trên xuống dễ hơn `.then()` lồng nhau.

```
async function layDuLieu(url) {
  try {
    const response = await fetch(url);   // Promise -> Response
    const data = await response.json();  // .json() cũng async
    return data;
  } catch (error) {
    console.error("Lỗi khi gọi API:", error);
  }
}
```

4. Fetch API, AJAX & JSON

- **AJAX** (Asynchronous JavaScript And XML): lấy dữ liệu từ server **không reload trang**. Trước đây dùng XMLHttpRequest / jQuery; nay dùng **Fetch API** và JSON thay cho XML.
- `JSON.parse(text)`: chuỗi JSON → object JS. `JSON.stringify(obj)`: object → chuỗi JSON. `response.json()` đã làm bước parse.

```
async function loadUsers() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const users = await response.json();
  const tbody = document.querySelector("#user-table tbody");
  users.forEach(user => {
    const row = document.createElement("tr");
    row.innerHTML = `<td>${user.id}</td><td>${user.name}</td>`;
    tbody.appendChild(row);
  });
}
loadUsers();
```

5. CORS

- **CORS** (Cross-Origin Resource Sharing): chính sách bảo mật của trình duyệt. Khi gọi API ở domain khác trang hiện tại, trình duyệt kiểm tra server có cho phép không; nếu không, request bị chặn (báo lỗi `blocked by CORS policy` trong Console).
- Cấu hình CORS phía server sẽ học cùng FastAPI (Week 10).

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
3 trạng thái của Promise?	pending, fulfilled, rejected
await dùng ở đâu?	Chỉ trong async function
Chuyển Response của fetch thành dữ liệu JS?	.json()
Cấu trúc bắt lỗi async?	try { } catch (error) { }
Vì sao JS đơn luồng cần async?	Tránh chặn UI khi chạy tác vụ chậm
Khi nào xảy ra lỗi CORS?	Trình duyệt chặn request tới domain không cho phép

Week 06 — Python Web & bắt đầu với FastAPI

Từ tuần này, backend là Python + FastAPI: gắn hàm Python vào URL.

1. Nhiệm vụ của server

- Client gửi HTTP request (verb + path, query params, headers, body) → server **route** (khớp verb + path với một hàm) → hàm xử lý, trả giá trị → framework đóng gói thành response (JSON, headers, status code).
- Với DS/AI: model ML là một hàm Python (input → prediction); web service là cách đưa hàm đó ra qua HTTP.

2. Virtual environment & packages

```
python -m venv .venv            # tạo môi trường riêng trong ./.venv
source .venv/bin/activate       # kích hoạt (macOS/Linux)
# .venv\Scripts\activate       # Windows
deactivate                      # thoát

pip install fastapi "uvicorn[standard]"
pip freeze > requirements.txt   # ghi lại gói + phiên bản
pip install -r requirements.txt # đồng đội cài lại y hệt
```

- venv tách biệt gói giữa các project. Cho `.venv/` vào `.gitignore`; commit `requirements.txt`, không commit môi trường.

3. WSGI vs ASGI, Uvicorn

	WSGI	ASGI
Mô hình	Đồng bộ	Bất đồng bộ
Dùng bởi	Flask, Django cổ điển	FastAPI (qua Starlette)
Khi chờ I/O	Worker bị chặn	Event loop phục vụ request khác
async def	Không hỗ trợ gốc	Hỗ trợ gốc

- **Uvicorn** là ASGI server chạy app (FastAPI không tự có server): `uvicorn main:app --reload`. `main` = file main.py, `app` = đối tượng FastAPI(), `--reload` **chỉ dùng khi dev**. Địa chỉ mặc định `http://127.0.0.1:8000`.

4. App FastAPI tối thiểu

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, Web!"}

@app.get("/hello/{name}")          # path parameter
def hello(name: str):
    return {"greeting": f"Hello {name}"}

@app.get("/add")                   # query params: /add?a=2&b=3
def add(a: int, b: int):
    return {"a": a, "b": b, "sum": a + b}
```

- `@app.get("/")` là path operation decorator; hàm bên dưới là path function.
- `{name}` trong path → path param; tham số không nằm trong path → query param.
- Type hint cho validation miễn phí: `/add?a=x&b=3` → lỗi **422** rõ ràng, không crash.
- Trả về dict/list/model → tự chuyển thành JSON, status mặc định 200.

5. def vs async def

- Dùng `async def` khi thân hàm cần `await`, ngược lại dùng `def`. FastAPI chạy `async def` trên event loop, `def` trong threadpool; cả hai đều không chặn. Không bao giờ tự gọi `asyncio.run()`.

6. Tài liệu tự động & cấu trúc project

- `/docs` là Swagger UI (có nút **Try it out**), `/redoc` là ReDoc, `/openapi.json` là mô tả OpenAPI thô, sinh tự động từ chữ ký hàm và type hint.

```
project/
├── frontend/          # HTML/CSS/JS từ Week 2-5
└── backend/
    ├── .venv/         # không commit
    ├── main.py        # FastAPI app
    └── requirements.txt
```

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
`python -m venv` làm gì?	Tạo môi trường Python riêng cho project
FastAPI cần chuẩn nào cho async def?	ASGI
Vai trò của Uvicorn?	ASGI server chạy app
`@app.get("/items")` khai báo gì?	Hàm bên dưới xử lý GET /items
Port mặc định của Uvicorn?	8000
Khi nào dùng --reload?	Chỉ khi phát triển
Hàm trả về dict → client nhận?	JSON, status 200

Week 07 — FastAPI: Routing & Request/Response

Làm cho API thực sự hoạt động: endpoint cho mọi verb, input từ path/query/body, output có định dạng, lỗi đúng chuẩn.

1. Path operations & HTTP verbs (bản đồ REST)

Decorator	Verb	CRUD	Status thành công
@app.get	GET	Read (lấy)	200
@app.post	POST	Create (tạo)	201
@app.put	PUT	Thay thế toàn bộ	200
@app.patch	PATCH	Sửa một phần	200
@app.delete	DELETE	Delete (xoá)	204

- Một resource có nhiều path: `/items`, `/items/{id}`. Tên resource là **danh từ số nhiều**; động từ là HTTP method, **không bao giờ nằm trong URL** (`/getItem` là kiểu RPC).

2. Path parameters

- Type hint quyết định chuyển kiểu + validation: `/items/5` → 5; `/items/abc` → **422**, hàm không chạy.
- Kiểu hỗ trợ: int, str, float, bool, Enum.
- **Thứ tự route quan trọng**: khai báo `/items/me` **trước** `/items/{item_id}` vì route nào khớp trước thì thắng.

3. Query parameters

```
from fastapi import Query

@app.get("/items")
def list_items(
    skip: int = Query(0, ge=0),
    limit: int = Query(10, ge=1, le=100),
    q: str | None = Query(None, min_length=2),
): ...
```

- Tham số scalar không có trong path → query param. Có default → tuỳ chọn; `str | None = None` → có thể vắng.
- `Query()` thêm ràng buộc (`ge`, `le`, `min_length`, `pattern`...) và metadata cho /docs.
- Query param dùng cho phần phụ (phân trang, lọc), **không dùng để định danh resource**.

4. Request body với Pydantic

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True        # có default -> tuỳ chọn

@app.post("/items")
def create_item(item: Item):
    return item
```

- Tham số kiểu BaseModel → FastAPI đọc JSON body, validate từng field, dựng object. Thiếu field/sai kiểu → **422** kèm danh sách lỗi theo field.
- BaseModel ≈ `@dataclass` + kiểm tra kiểu lúc chạy + (de)serialize. Có thể trộn path + query + body trong một hàm.

5. response_model, status_code & HTTPException

```
class ItemCreate(BaseModel):   # client gửi lên
    name: str; price: float

class ItemPublic(BaseModel):   # client được thấy
    id: int; name: str; price: float

@app.post("/items", response_model=ItemPublic, status_code=201)
def create_item(item: ItemCreate):
    saved = {"id": 1, "name": item.name, "price": item.price, "secret": "xyz"}
    return saved               # "secret" bị loại vì không có trong ItemPublic

@app.get("/items/{item_id}", response_model=ItemPublic)
def get_item(item_id: int):
    item = _find(item_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return item
```

- `response_model` lọc/định dạng output theo schema công khai, ẩn field thừa, hiển thị trên /docs. Nên **tách model input và output**.
- `HTTPException` → JSON sạch `{"detail": "Item not found"}`. Mã hay dùng: 400, 401, 403, 404, 409.

6. Static files & templates

```
from fastapi.staticfiles import StaticFiles
app.mount("/static", StaticFiles(directory="static", html=True), name="static")
```

- `StaticFiles` phục vụ thư mục `frontend/` từ backend, cùng origin nên không bị CORS khi dev; `fetch()` dùng URL tương đối được.
- Jinja2 templates (`{{ }}`, `{% %}`) render HTML phía server, không bắt buộc với kiến trúc API + SPA tĩnh.

7. Lab 4 mở rộng (Extended Lab)

Phần	Nội dung chính
A — PATCH	`ItemUpdate` mọi field tuỳ chọn; `model_dump(exclude_unset=True)` chỉ lấy field client gửi. PUT = trạng thái đầy đủ mới; PATCH = chỉ sửa field được gửi.
B — Lọc/tìm/sắp xếp	`min_price`, `max_price`, `q`, `sort_by` (`pattern="^(id|name|price)$"`), `order`. **Lọc + sắp xếp TRƯỚC, cắt skip/limit SAU.**
C — 409 Conflict	Tên trùng (không phân biệt hoa/thường) → `HTTPException(409)`; kiểm tra cả khi đổi tên bằng PUT/PATCH.
D — Envelope	`ItemListResponse {items, total, skip, limit}`; `total` = số item sau khi lọc, trước khi cắt trang.
E — Predict	`POST /predict/house-price`; ràng buộc body bằng `Field(gt=0)` / `Field(ge=0)` → 422 tự động.

Ôn tập trắc nghiệm

Câu hỏi	Đáp án
`/items/{id}?limit=10`: đâu là path, đâu là query?	id: path, limit: query
Tham số BaseModel được hiểu là?	JSON request body, tự validate
Status code khi tạo resource?	201 Created
Field trong body sai kiểu → ?	422 Unprocessable Entity
Mục đích của response_model?	Lọc/định dạng output theo schema công khai
HTTPException dùng để?	Trả lỗi có kiểm soát (code + detail)
Vì sao `/items/me` phải đứng trước `/items/{id}`?	Route khớp theo thứ tự; nếu không "me" bị coi là id